

TRAJCENTER v2.0 — Protocole de communication PC ↔ Robot

Version : 2.0

Date : 2026-07-08

Auteurs : J. SCHUMACKER / C. RACINET

RobotWare : 6.x

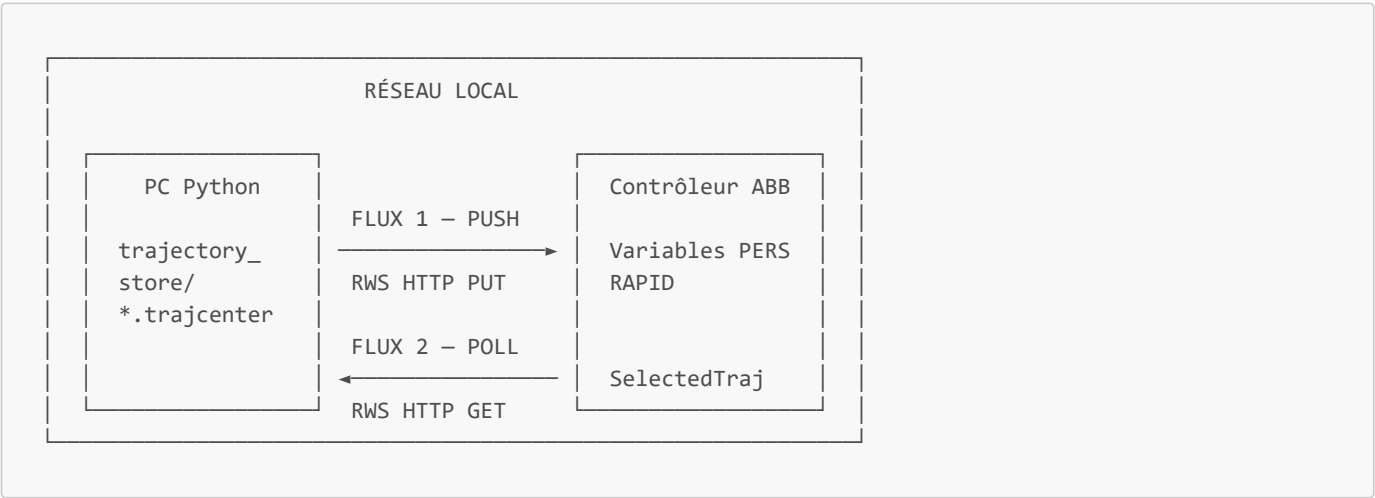
1. Vue d'ensemble

1.1 Principe général

TrajCenter v2.0 repose sur un **unique protocole de transport : HTTP REST via ABB Robot Web Services (RWS)**. Le PC Python et le contrôleur robot communiquent exclusivement par requêtes HTTP sur le réseau local. Il n'y a plus de serveur TCP custom côté PC.

Le PC est **actif** : c'est lui qui écrit les données dans les variables RAPID du contrôleur. Le robot est **passif** : il lit des variables déjà remplies et signale ses intentions via une variable RAPID que Python surveille.

1.2 Les deux flux de communication



Flux	Direction	Mécanisme	Contenu
FLUX 1 — PUSH	PC → Robot	PUT RWS	Métadonnées, robtargets, tools, wobjs
FLUX 2 — POLL	Robot → PC (lecture)	GET RWS polling 200 ms	SelectedTrajIndex

1.3 Ce qui disparaît par rapport à la v1

v1	v2
Serveur TCP Python (port 50000)	Supprimé
RAPID client TCP (SocketCreate, SocketConnect)	Supprimé
Encodage binaire INT32 little-endian	Supprimé
Requêtes texte <code>robt[i;j;k], nbtraj...</code>	Supprimées
Réception par paquets de 15 robtargets	Supprimée

2. Variables RAPID — Module système TRAJCENTER.sys

Toutes les variables partagées entre Python et RAPID sont déclarées dans le module système **TRAJCENTER.sys** avec le mot-clé **PERS**. Ce mot-clé est **obligatoire** pour qu'une variable soit accessible en lecture et en écriture via RWS.

Le module système est chargé au démarrage du contrôleur, avant tout module programme. Les variables sont donc disponibles immédiatement et persistent entre les arrêts/redémarrages du programme RAPID.

2.1 Tableau complet des variables PERS

Variable	Type RAPID	Taille	Écrit par	Lu par	Rôle
NbTrajDispo	num	4 B	Python	RAPID	Nombre de trajectoires disponibles dans le store
NomsTraj{50}	string{50}	~1.5 KB	Python	RAPID	Noms des trajectoires disponibles (index 1..N)
NbPointsTraj{50}	num{50}	200 B	Python	RAPID	Nombre de points par trajectoire
SelectedTrajIndex	num	4 B	RAPID	Python	Index de la trajectoire choisie par l'opérateur
NbRobtargetsTraj	num	4 B	Python	RAPID	Nombre de points de la trajectoire chargée
RobtTRAJCENTER{100000}	robtaraget{100000}	~7.8 MB	Python	RAPID	Tableau des robtargets de la trajectoire chargée
NbTool	num	4 B	Python	RAPID	Nombre de tools nécessaires pour la trajectoire
ToolNames{10}	string{10}	~300 B	Python	RAPID	Noms des tools (index 0..NbTool-1)
NbWobj	num	4 B	Python	RAPID	Nombre de wobjs nécessaires pour la trajectoire
WobjNames{10}	string{10}	~300 B	Python	RAPID	Noms des wobjs (index 0..NbWobj-1)
TrajReady	bool	1 B	Python	RAPID	Signal : transfert terminé, trajectoire prête
RefreshFlag	bool	1 B	Python	RAPID	Signal : liste des trajectoires mise à jour

Empreinte mémoire totale des métadonnées : ~2.3 KB Empreinte mémoire RobtTRAJCENTER{100000} : ~7.8 MB Total : ~8 MB sur 39 MB disponibles

2.2 Convention d'indexation

- NomsTraj, NbPointsTraj, RobtTRAJCENTER : indexés à **partir de 1** (convention RAPID)
- ToolNames, WobjNames : indexés à **partir de 0** (index de la colonne `tool_index` / `wobj_index` du fichier `.trajcenter`)
- SelectedTrajIndex : valeur **1..NbTrajDispo** — 0 signifie "aucune sélection en cours"

3. Format d'un robtaraget RWS

3.1 Structure

Un robtaraget RAPID contient 17 valeurs numériques organisées en quatre groupes :

```
[[x, y, z], [q1, q2, q3, q4], [cf1, cf4, cf6, cfx], [eax_a, eax_b, eax_c, eax_d, eax_e, eax_f]]
```

Groupe	Champs	Unité	Description
trans	x, y, z	mm	Position TCP
rot	q1, q2, q3, q4	—	Orientation (quaternion, convention ABB : scalaire en premier)
robconf	cf1, cf4, cf6, cfx	entier	Configuration des axes

Groupe	Champs	Unité	Description
extax	eax_a ... eax_f	mm ou deg	Axes externes

3.2 Convention axes externes inactifs

Un axe externe inactif (non présent sur le robot) est représenté par la valeur **9E9** exactement dans le robtarget RAPID. Cette valeur est injectée par Python au moment de la sérialisation — elle n'est **pas** stockée dans le fichier **.trajcenter** (les colonnes absentes du fichier signifient axe inactif).

3.3 Convention quaternions ABB

ABB utilise la convention **scalaire en premier** : `[w, x, y, z]` soit `[q1, q2, q3, q4]` dans la nomenclature RAPID. Cette convention doit être respectée dans tous les convertisseurs.

4. Routes RWS

4.1 Authentification

Toutes les requêtes RWS utilisent :

- **Authentification** : HTTP Digest
- **Credentials par défaut** : `Default User / robotics`
- **Cookie de session** : `ABBCX` (maintenu entre les requêtes)
- **Format de réponse** : JSON (`?json=1` en paramètre de requête)
- **Base URL** : `http://<IP_ROBOT>/rw/`

4.2 Tableau des routes utilisées

Lecture (GET) — Python interroge le contrôleur

ID	Route	Description	Réponse
G1	<code>GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SelectedTrajIndex</code>	Lit l'index de trajectoire choisi par l'opérateur	<code>num</code>
G2	<code>GET /rw/rapid/execution</code>	Lit l'état d'exécution du programme RAPID	état contrôleur

Écriture (PUT) — Python écrit dans le contrôleur

L'écriture de variables RAPID via RWS nécessite d'avoir acquis le **Mastership RAPID** au préalable (voir section 5).

ID	Route	Corps de la requête	Description
W1	<code>PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbTrajDispo</code>	<code>value=N</code>	Nombre de trajectoires disponibles
W2	<code>PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NomsTraj</code>	<code>value=["traj1", "traj2", ...]</code>	Noms des trajectoires
W3	<code>PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbPointsTraj</code>	<code>value=[1500, 320, ...]</code>	Nombre de points par trajectoire
W4	<code>PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbRobtargetsTraj</code>	<code>value=N</code>	Nombre de points de la trajectoire

ID	Route	Corps de la requête	Description
			en cours de transfert
W5	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RobtTRAJCENTER/[i]	value=[x,y,z], [q1,q2,q3,q4], [cf1,cf4,cf6,cfx], [eax_a,...,eax_f]	Écriture du robtarget à l'index i (boucle sur tous les points)
W6	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbTool	value=N	Nombre de tools de la trajectoire chargée
W7	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/ToolNames	value=["tool0","tool1",...]	Noms des tools
W8	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbWobj	value=N	Nombre de wobjs de la trajectoire chargée
W9	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/WobjNames	value=["wobj0","wobj1",...]	Noms des wobjs
W10	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TrajReady	value=TRUE	Signal de fin de transfert
W11	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RefreshFlag	value=TRUE	Signal de mise à jour de la liste

Mastership (POST)

ID	Route	Description
M1	POST /rw/mastership/request	Acquisition du Mastership RAPID
M2	POST /rw/mastership/release	Libération du Mastership RAPID

5. Mastership RAPID

5.1 Principe

En RW6, **toute écriture de variable RAPID via RWS nécessite le Mastership**. C'est un verrou exclusif — un seul client peut l'avoir à la fois. Il doit être acquis avant la première écriture et libéré dès que possible.

5.2 Contraintes

- Le Mastership est **refusé** si le programme RAPID est en cours d'exécution en mode automatique
- Il doit être **toujours libéré** même en cas d'erreur (utiliser un context manager)
- Ne pas conserver le Mastership plus longtemps que nécessaire

5.3 Séquence d'écriture

```
POST /rw/mastership/request      → acquisition
  PUT W4 NbRobttargetsTraj
  PUT W6 NbTool
  PUT W7 ToolNames
  PUT W8 NbWobj
  PUT W9 WobjNames
  PUT W5 RobtTRAJCENTER[1]
  PUT W5 RobtTRAJCENTER[2]
  ...
  PUT W5 RobtTRAJCENTER[N]
  PUT W10 TrajReady = TRUE
POST /rw/mastership/release      → libération
```

6. Pipeline complète

6.1 Phase 1 — Démarrage Python

Déclenchée au lancement du processus Python.

1. Scan du trajectory_store/ → liste des fichiers .trajcenter
2. Pour chaque fichier : lecture de meta.json (name, point_count)
3. PUT W1 : NbTrajDispo = N
4. PUT W2 : NomsTraj = [nom1, nom2, ..., nomN]
5. PUT W3 : NbPointsTraj = [nb1, nb2, ..., nbN]
6. Démarrage de la boucle de polling (GET G1 toutes les 200 ms)
7. Démarrage du watchdog trajectory_store/ (scan toutes les 5 s)

6.2 Phase 2 — Sélection opérateur (côté RAPID)

L'opérateur interagit avec le FlexPendant. RAPID lit les variables déjà en mémoire — aucune requête réseau nécessaire à cette étape.

- RAPID :
1. Affichage du menu FlexPendant avec NomsTraj[1..NbTrajDispo]
 2. Opérateur sélectionne la trajectoire i
 3. SelectedTrajIndex := i
 4. WaitUntil TrajReady = TRUE \MaxTime:=120

6.3 Phase 3 — Détection et transfert (côté Python)

- Python (boucle polling) :
1. GET G1 : SelectedTrajIndex → détecte changement (valeur ≠ 0 et ≠ dernière valeur)
 2. Charge le fichier .trajcenter[i] depuis trajectory_store/
 3. Acquisition Mastership (POST M1)
 4. PUT W4 : NbRobttargetsTraj = N
 5. PUT W6/W7 : NbTool + ToolNames
 6. PUT W8/W9 : NbWobj + WobjNames
 7. Pour j = 1 à N :
 - PUT W5 : RobtTRAJCENTER[j] = sérialisation du point j
(axes externes inactifs → 9E9)
 8. PUT W10 : TrajReady = TRUE
 9. Libération Mastership (POST M2)

6.4 Phase 4 — Exécution (côté RAPID)

```

RAPID :
1. WaitUntil TrajReady = TRUE → débloqué
2. TrajReady := FALSE
3. Vérification NbTool, NbWobj (cohérence avec tools/wobjjs déclarés)
4. Lancement TRAJCENTER_Move :
    FOR i FROM 1 TO NbRobtTargetsTraj DO
        MoveL/MoveJ RobtTRAJCENTER{i}, speed, zone, tool, wobj
    ENDFOR
    
```

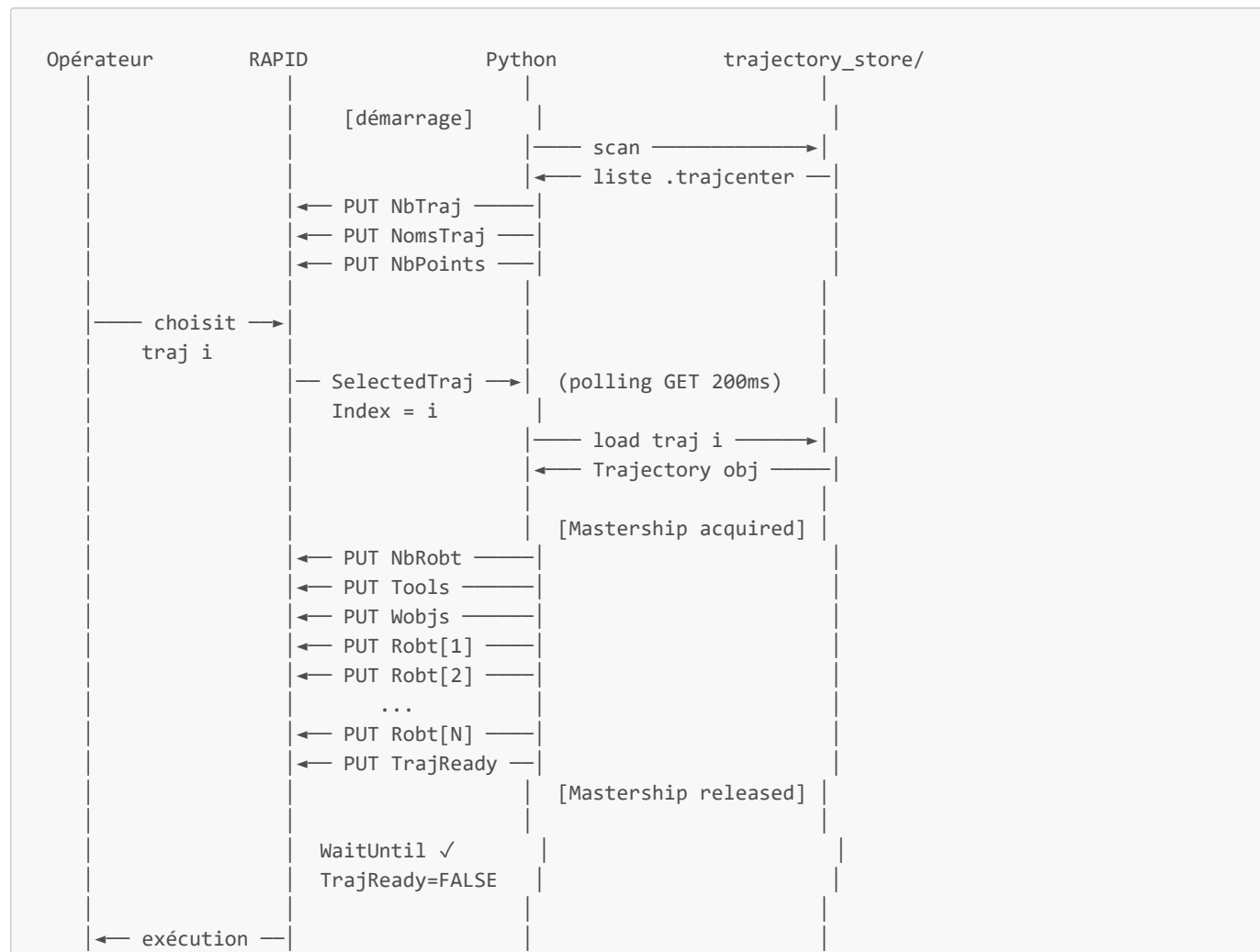
6.5 Phase 5 — Refresh à chaud

```

Python (watchdog) :
1. Détecte nouveau/supprimé .trajcenter dans trajectory_store/
2. Re-scanner la liste
3. Acquisition Mastership
4. PUT W1/W2/W3 : mise à jour NbTrajDispo, NomsTraj, NbPointsTraj
5. PUT W11 : RefreshFlag = TRUE
6. Libération Mastership

RAPID (tâche background) :
1. WaitUntil RefreshFlag = TRUE
2. Rafraîchit l'affichage FlexPendant
3. RefreshFlag := FALSE
    
```

7. Diagramme de séquence complet



	TRAJCENTER			
	_Move			

8. Gestion des erreurs

Situation	Comportement Python	Comportement RAPID
Mastership refusé (robot en auto)	Retry × 3 puis log erreur	—
Timeout transfert (> 120 s)	Log erreur, libère Mastership	WaitUntil \TimeFlag → gestion timeout
Fichier .trajcenter corrompu	Log erreur, ne transfère pas, SelectedTrajIndex remis à 0	Reste en attente
Perte réseau pendant transfert	Exception HTTP, libère Mastership (finally)	Reste en WaitUntil jusqu'au timeout
Index hors bornes ($i > NbTrajDispo$)	Ignoré, log warning	—

9. Paramètres de configuration

Paramètre	Valeur par défaut	Description
ROBOT_IP	—	Adresse IP du contrôleur (.env, non versionné)
RWS_PORT	80	Port HTTP RWS
RWS_USER	Default User	Identifiant RWS
RWS_PASSWORD	robotics	Mot de passe RWS (.env, non versionné)
POLL_INTERVAL_S	0.2	Intervalle de polling SelectedTrajIndex (secondes)
WATCHDOG_INTERVAL_S	5.0	Intervalle de scan du trajectory_store/ (secondes)
MASTERSHIP_RETRY	3	Nombre de tentatives d'acquisition du Mastership
TRAJ_READY_TIMEOUT_S	120	Timeout côté RAPID pour WaitUntil TrajReady
MAX_TRAJ	50	Taille du tableau NomsTraj / NbPointsTraj
MAX_TOOLS	10	Taille du tableau ToolNames
MAX_WOBS	10	Taille du tableau WobjNames
MAX_ROBTARGETS	100000	Taille statique de RobtTRAJCENTER

10. Estimation mémoire RAPID

$$\text{Mémoire}(N) = N \times 17 \times 4 \text{ bytes} = N \times 68 \text{ bytes}$$

Tableau déclaré	Mémoire brute	Avec overhead (~15%)
RobtTRAJCENTER{10000}	680 KB	~780 KB
RobtTRAJCENTER{50000}	3.4 MB	~3.9 MB
RobtTRAJCENTER{100000}	6.8 MB	~7.8 MB
Métadonnées totales	~2.3 KB	~2.3 KB
Total {100000} + métadonnées	~6.8 MB	~8 MB / 39 MB